

הנדסה לאחור - מטלה 1

מיכאל בייגל 213764251 דניאל יהושע 215332362

חלק ב

ראשית, קובץ ההרצה המקורי פועל כך: הוא מגריל בתחילה מספר אקראי בין 1 ל-100, כקלט הוא מקבל בכל פעם את הערך שהמשתמש חושב שהוא שווה אליו, אם הערך בקלט לא שווה לערך המוגרל הוא יבקש מהמשתמש לנסות שנית (ויגיד אם ערך הקלט גדול או קטן מהערך המוגרל), עד שהתוצאה הנכונה מתקבלת ואז הוא יוצא.

1. אופטימיזיות - partb.opt.s

- תיקון ראשון:
אנחנו מדפסים פעמיים את "Guess a number between 1 and 100" אז נחליף את ההדפסה השניה להיות "enter your guess".
- תיקון שני:
בקטע L2 אם המספר שהוגרל גדול שווה מהמספר שהכניס המשתמש אז היינו קופצים חזרה לתווית L2 במקום לקטע הקוד שבודק כעת אם המספר שהוגרל קטן שווה מהמספר שהכניס המשתמש.
- תיקון שלישי:
אך כעת נשים לב לכך שהרצה של הקוד במצב כזה תמיד תיתן שהמספר המוגרל יהיה שווה ל-42, מה שקורה כי אנו לא קוראים ל-srand בקוד, ניתן לפתור זאת ע"י קריאה ל-srand (אם זמן כקלט) לפני הקריאה ל-rand.

2. ללא אופטימיזיות - partb.nopt.s

- תיקון ראשון:
במידה וקלט המשתמש קטן\גדול מהמספר המוגרל אז נדפיס "המספר קטן\גדול מידי" ונקפוצ לסוף התוכנית במקום לנסות שוב, עלינו לקפוצ חזרה למקום בו לוקחים קלט מן המשתמש.
- תיקון שני:
לא העברנו פרמטרים אל הפונקציה scanf, עלינו להעביר את הטיפוס שהפונקציה תצפה לקבל מהמשתמש ובנוסף הכתובת אליה יכנס הקלט מן המשתמש.

חלק ג

מסך ראשון:login

בשלב זה ביצענו inspect element וגילינו שיש אלמנט מהמחלקה "button disabled" שלחיצה עליו מפעילה את הפונקציה challenge_me. כעת שינינו את מחלקת האלמנט לbutton, מה שהחזיר אותו ואפשר לנו ללחוץ עליו, מה שהעביר אותנו לשלב הבא

מסך שני:challenge

כעת, ראינו שנדרשנו לבנות תוכנה שתחשב את מספר המשאבים הדרושים מכל סוג ותפלוט אותו בהינתן קלט של דברים שנרצה לבנות במשחק "קטאן". לאחר שכתבנו אותה, גילינו בהגשה שגודל הקובץ צריך להיות קטן מ4k. לכן נעזרנו בשיטות שלמדנו בתרגול 2 כדי להקטין את גודל קובץ ההגשה למימדים הדרושים.

מסך שלישי:recovery

כעת, הורדנו את קובץ ההרצה וקובץ המקור של crackme וניגשנו לעבוד על השלבים הבאים:

שלב 1

```
PS C:\Users\mlovets\Technion\RE\HW1\partC> ./crackme.exe
PS C:\Users\mlovets\Technion\RE\HW1\partC> ./crackme.exe 1
Level 1 Passed!
```

מספר הפרמטרים המועברים בריצה כולל הpath של הקובץ צריך להיות גדול מ1.

שלב 2

```
PS C:\Users\mlovets\Technion\RE\HW1\partC> ./crackme.exe 1
Level 1 Passed!
0 9
81591832 E1DF54FB 874756BF
49000000
164F21CA
DE5F64CD
Level 2 Passed!
PS C:\Users\mlovets\Technion\RE\HW1\partC>
```

נשים לב כי על מנת לעבור את שלב 2, נצטרך שמערך של 9 בתים יהיה שווה למערך של אותיות רנדומלי.
ונשים לב שprintArray בהינתן 2 מספרים שנכניס כקלט מדפיסה קבוצות של 4 בתים הנמצאות בטווח הבתים הזה החל מתחילת המערך הרנדומלי(האמת שזה לא באמת מתחילתו אלא קצת לפניו בגלל alignment).
כלומר אנחנו יכולים לדעת את תוכן המערך הרנדומלי.
כעת אנחנו מתבקשים להכניס קבוצות של 4 בתים ככמות שהודפס.
ומבוצע xor בין input שלנו ותוכן הזיכרון שהודפס ומיד נכתב בחזרה אליו.
כלומר ברשותנו לקרוא כמה בתים שנרצה בערך מתחילת המערך הרנדומלי שבמחסנית וגם לשכתב את אותם ערכים בעזרת חישוב xor.
כעת נשכתב את ערכי המערך הרנדומלי כך שיתאימו למערך הידוע בזיכרון.

שלב 3

```
PS C:\Users\mlovets\Technion\RE\Hw1\partC> ./crackme.exe 1
Level 1 Passed!
0 45
81591832 E1DF54FB 874756BF 0000002D
00000000 006BFF18 006BFEE8 006BFF10
00401841 00000009 006BFF28 004019CD

49000000 164F21CA DE5F64CD 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000EBC
Level 2 Passed!
Level 3 Passed!
```

נשים לב שבשביל לעבור את שלב 3 עלינו להגיע לתווית dummy_ אך אליה אין כלל קפיצות, מכאן ומהיכולות שתוארו בשלב 2(קריאה ושכתוב ערכים במחסנית) נוכל כעת לשכתב את ערך החזרה מהפונקציה level_2 אל dummy.

שלב 4

Download Recovery Source

Download Recovery Executable

Command line arguments

5 1

Input:

```
0 69
B0000000 1E3C495D CBB15DAB 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000542
```

Output:

```
Level 1 Passed!
78591832 E9AC3C6C 92A96FD9 00000045
00000000 006BFF30 006BFEE8 775BB119
00401841 00000009 006BFF28 004019CD
68927480 00732A78 00000005 00401771
006BFF80 00401233
Level 2 Passed!
Level 3 Passed!
Level 4 Passed!
```

Errors:

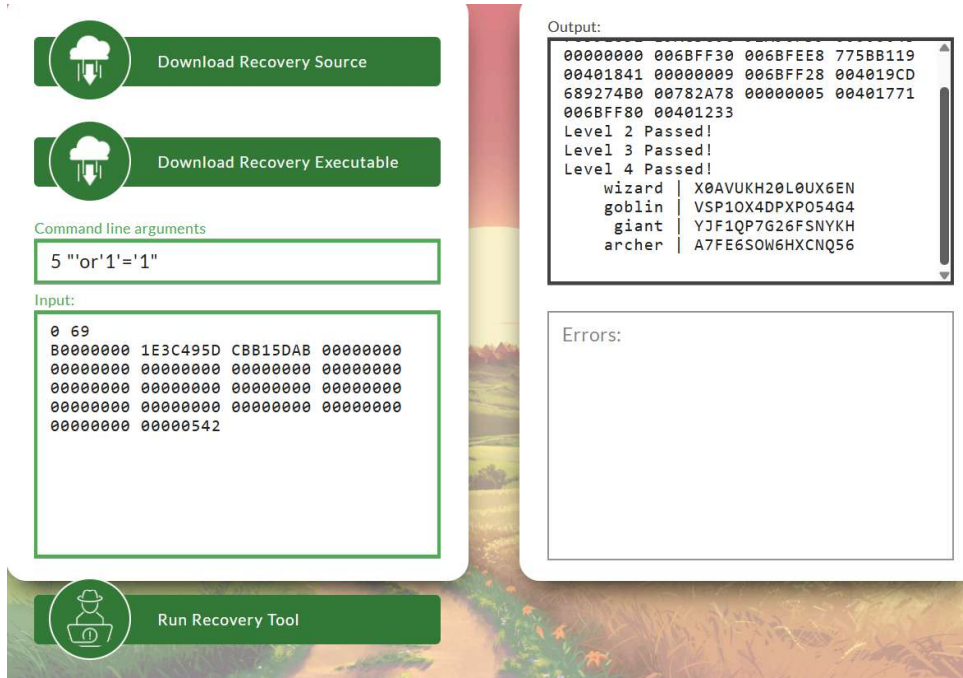
Run Recovery Tool

שמנו לב שעל מנת לעבור את שלב 4 עלינו לקפוץ אל handler , שאותה ייעדנו במain להיות הפונקציה שתטפל בסיגנל של חלוקה ב0. כעת שמנו לב שהמקום היחיד בו מבוצעת חלוקה כלשהי הוא ב dummy מיד לאחר שעוברים את שלב 3. בה אנו מחלקים ערך מספרי קבוע השמור ב edx:eax בערך הנמצא בתווית divider. לבסוף, גילינו שיש מקום יחיד בו נוכל לשכתב את הערך בdivider, והוא נמצא בסוף הפונקציה main, ממש לפני חזרתה. לכן הסקנו שעלינו לשנות את כתובת החזרה מmain אל dummy במקום מlevel_2 אל dummy כדי לאפשר כתיבה אל divider, ולשנות את הערך שנקבל מrand_32 כך שבסיום אותם חישובים, אנו נכניס לdivider את הערך 0.

בשביל לעשות זאת, ניסינו מספר הכנסות שונות של הארגומנט הראשון, על מנת למצוא seed שיאפס את התוצאה המתקבלת מהחישוב המתבצע בסוף main ונעזר בערכי rand_32 , עד שמצאנו שהערך 5 מקיים את הדרישה. דבר זה מכניס ערך 0 לdivider, וכעת, במקום לשנות את כתובת החזרה מlevel_2 , שינינו את כתובת החזרה מmain , כך שהיא תהיה ההתחלה של dummy , וכעת בתוך dummy יש חלוקה באפס (שמשמשת בdivider כמכנה) כך שנגיע לhandler שמטפל בסיגנל של חלוקה באפס, ובו ניתן לעבור את שלב 4.

ה handler מטפל בזה וחוזר ל dummy עם divider שונה מ0. וממשיך אל db_access.

פריצה סופית למאגר הנתונים:



ה handler מטפל בזה וחוזר ל dummy עם divider שונה מ0. וממשיך אל db_access.

כעת כאשר עברנו את כלל השלבים, שמנו לב שה handler מטפל בשגיאה וחוזר ל dummy עם divider שונה מ0 (בעזרת מנגנון `longjmp setjmp`). וממשיך אל db_access. ראינו שבאותה פונקציה יש request למאגר sql הכתובה כך:

```
select username, password from users where username='{second arg}'
```

כאשר second arg הוא הארגומנט השני בריצתנו (`crackme.exe first_arg second_arg`) וכעת שמנו לב שנוכל לבצע הזרקת sql על ידי שינוי ה request:

```
select username, password from users where username="OR'1'='1'
```

כלומר הארגומנט השני יהיה `OR'1'='1'`
או ב CMD יהיה `"OR'1' = '1"`

